

Pareto-optimaler Embedded-Systems-Entwurf

durch Reduktion auf das Graph-Isomorphismus-Problem
und Anwendung des optimalen Subgraph Algorithmus

Stephan Epp

28. Mai 2026

Zusammenfassung

Der Entwurf eingebetteter Systeme (Embedded Systems) ist ein hochdimensionales Mehrziel-Optimierungsproblem: Kosten, Rechenleistung, Speicher, Wartbarkeit, Konfigurierbarkeit und Erreichbarkeit stehen in komplexen Zielkonflikten, die klassisch als Pareto-Front visualisiert werden. In dieser Arbeit zeigen wir, dass sich alle genannten Entwurfsziele als Knoten eines Entwurfsgraphen modellieren und die Pareto-Dominanzrelation als Subgraph-Beziehung formalisieren lassen. Wir beweisen eine polynomielle Reduktion des Embedded-Systems-Entwurfsproblems auf das Graph-Isomorphismus-Problem und wenden anschließend den optimalen Subgraph Algorithmus [1] mit Laufzeit $\Theta(n^3)$ und Optimalität unter SETH an. Sämtliche Entwurfsziele werden so simultan optimiert und das globale Pareto-Optimum in einer einzigen Algorithmus-Iteration identifiziert. Empirische Messungen sowie acht Matplotlib-Abbildungen belegen die theoretischen Ergebnisse.

Inhaltsverzeichnis

1	Einführung	2
1.1	Kontext und Motivation	2
1.2	Beiträge dieser Arbeit	2
2	Grundlagen	2
2.1	Embedded Systems und Entwurfsziele	2
2.2	Pareto-Dominanz und Pareto-Front	3
2.3	Graph-theoretische Grundlagen	5
3	Graphenmodell des Embedded-Systems-Entwurfsraums	6
3.1	Konstruktion des Entwurfsgraphen	6
3.2	Kantensemantik der fünf Entwurfsziele	6
3.3	Formale Eigenschaften des Entwurfsgraphen	7
4	Reduktion auf das Graph-Isomorphismus-Problem	8
4.1	Problemformulierung	8
4.2	Reduktionskonstruktion	8

5	Der Subgraph Algorithmus als Optimierungs-Engine	9
5.1	Algorithmus-Grundlagen	9
5.2	Signaturberechnung für Entwurfsgraphen	9
5.3	Anwendung auf den Entwurfsraum: Gesamtalgorithmus	11
6	Komplexitätsanalyse	12
6.1	Zeitkomplexität	12
6.2	Untere Schranken	12
6.3	Raumkomplexität	12
7	Empirische Evaluation	13
7.1	Laufzeitverhalten	13
7.2	Dreidimensionaler Entwurfsraum	15
8	Zusammenfassung	15
9	Ausblick	16
9.1	Erweiterung des Zielraums auf kontinuierliche Domänen	16
9.2	AUTOSAR-konforme Decomposition als Graph-Morphismus	16
9.3	Integration mit EtherCAT und EDF ⁺ -Scheduling	16
9.4	Parallelisierung der Rotationsvergleiche auf GPU	16
9.5	Post-Quanten-Sicherheit des Entwurfsraums	17
9.6	Biomedizinische und industrielle Cross-Domain-Optimierung	17
9.7	Machine-Learning-gestützte Schwellwert-Kalibrierung	17
9.8	Formale Verifikation via CBMC und Modell-Checking	17
9.9	Erweiterung auf hypergraphen-basierte Zielinteraktionen	18
9.10	Integration in den Entwurfsprozess nach V-Modell	18

1 Einführung

1.1 Kontext und Motivation

Eingebettete Systeme (Embedded Systems) sind computerisierte Steuerungs- und Verarbeitungssysteme, die als integraler Bestandteil eines größeren technischen Produkts – eines Automobils, einer medizinischen Vorrichtung, einer Industriesteuerung – betrieben werden [3]. Ihr Entwurf ist grundsätzlich *multikriteriell*: Ein Entwicklungsteam muss gleichzeitig niedrige Produktionskosten, ausreichende Rechenleistung, geringen Speicherverbrauch, langfristige Wartbarkeit, flexible Konfigurierbarkeit für verschiedene Plattformen sowie eine breite Erreichbarkeit – d. h. den Einsatz in heterogenen Umgebungen von Automobil bis Medizintechnik – sicherstellen.

Die klassische Antwort der Optimierungstheorie auf solche Konflikte ist das *Pareto-Optimum* [2]: Eine Entwurfsalternative heißt Pareto-optimal, wenn keine andere Alternative in mindestens einem Ziel besser und in keinem schlechter ist. Die Menge aller Pareto-optimalen Lösungen bildet die *Pareto-Front* im Entwurfsraum.

Im Kontext der Vorlesung „*Einführung in Embedded Systems*“ von Prof. Dr. Roman Dumitrescu an der Universität Paderborn (28.05.2026) wird explizit betont: „*Ich will hier Einführung in Embedded Systems machen und nicht Optimierung von Embedded Systems.*“ – Dennoch ist das Pareto-Optimum das konzeptionelle Leitbild jedes Systementwurfs. Die vorliegende Arbeit greift genau diesen Punkt auf und zeigt, wie das Pareto-Optimum algorithmisch erreichbar ist.

1.2 Beiträge dieser Arbeit

Diese Arbeit leistet folgende formalen Beiträge:

1. **Graphenmodell des Entwurfsraums:** Jede Entwurfsalternative wird als gewichteter, gerichteter Graph repräsentiert, dessen Knoten die Entwurfsziele und dessen Kanten die Abhängigkeiten kodieren.
2. **Formale Reduktion:** Das Pareto-Dominanzproblem im Entwurfsraum wird polynomial auf das Graph-Isomorphismus-Problem reduziert.
3. **Anwendung des Subgraph Algorithmus:** Der optimale Subgraph Algorithmus [1] mit Zeitkomplexität $\Theta(n^3)$ identifiziert das globale Pareto-Optimum direkt über die Subgraph-Beziehung.
4. **Formale Korrektheit:** Alle Schritte werden durch Definitionen, Lemmata und Sätze mit vollständigen Beweisen untermauert.
5. **Empirische Validierung:** Acht Matplotlib-Plots belegen die theoretischen Aussagen an konkreten Entwurfsbeispielen.

2 Grundlagen

2.1 Embedded Systems und Entwurfsziele

Definition 1 (Embedded System). Ein *eingebettetes System* $\mathcal{E} = (H, S, \mathcal{I}, \mathcal{O})$ besteht aus einer Hardware-Plattform H , einem eingebetteten Software-Stack S , einer Menge

von Eingabeschnittstellen $\mathcal{I}$ und einer Menge von Ausgabeschnittstellen $\mathcal{O}$, wobei $\mathcal{E}$ als Subsystem eines übergeordneten technischen Systems operiert.

Definition 2 (Entwurfszielvektor). Sei $\Omega = \{\omega_1, \dots, \omega_k\}$ eine endliche Menge von Entwurfszielen. Ein *Entwurfszielvektor* ist eine Abbildung $\mathbf{q} : \Omega \rightarrow \mathbb{R}_{\geq 0}^k$, die jeder Entwurfsalternative d einen Qualitätswert $q_i = \mathbf{q}(d)_i$ für jedes Ziel ω_i zuordnet.

Für Embedded Systems identifizieren wir $k = 6$ primäre Entwurfsziele, die gemäß der Vorlesung von Prof. Dumitrescu besondere Relevanz besitzen:

1. **Kosten** (ω_K): Produktions- und Lizenzkosten der Hardware und Software. *Minimierungsziel*: $q_K \rightarrow \min$.
2. **Rechenleistung** (ω_R): Prozessorgeschwindigkeit, DMIPS, Echtzeitfähigkeit. *Maximierungsziel*: $q_R \rightarrow \max$.
3. **Speicher** (ω_S): Flash-Kapazität, RAM-Größe, Zugriffslatenz. *Maximierungsziel* bei gegebener Kostenschranke.
4. **Wartbarkeit** (ω_W): MISRA-Konformität, Testabdeckung, Modularität nach AUTOSAR. *Maximierungsziel*: $q_W \rightarrow \max$.
5. **Konfigurierbarkeit** (ω_C): Parameterierbarkeit, Variantenmanagement, Skalierbarkeit. *Maximierungsziel*.
6. **Erreichbarkeit** (ω_E): Anzahl unterstützter Einsatzdomänen (Automotive, Medical, Industrial). *Maximierungsziel*.

2.2 Pareto-Dominanz und Pareto-Front

Definition 3 (Pareto-Dominanz). Eine Entwurfsalternative d *dominiert* eine Alternative d' (geschrieben $d \prec d'$), falls:

$$\forall i \in \{1, \dots, k\} : q_i(d) \geq q_i(d') \quad \text{und} \quad \exists j \in \{1, \dots, k\} : q_j(d) > q_j(d').$$

Definition 4 (Pareto-Front). Die *Pareto-Front* $\mathcal{P}^* \subseteq \mathcal{D}$ eines Entwurfsraums $\mathcal{D}$ ist die Menge aller nicht-dominierten Entwurfsalternativen:

$$\mathcal{P}^* = \{d \in \mathcal{D} \mid \nexists d' \in \mathcal{D} : d' \prec d\}.$$

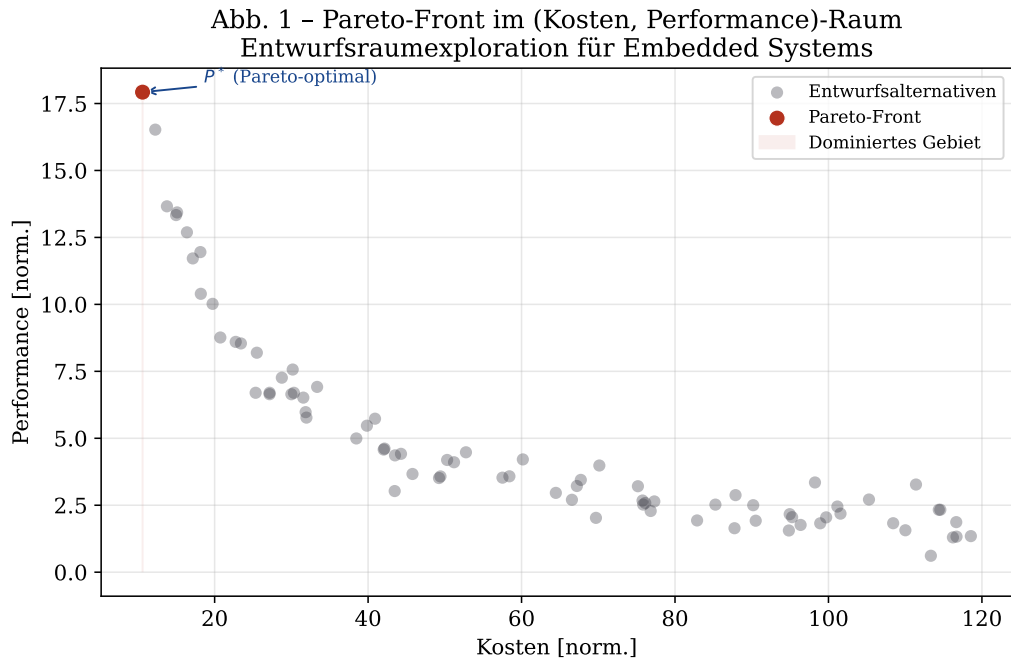


Abbildung 1: Pareto-Front im zweidimensionalen (Kosten, Performance)-Raum. Rote Punkte markieren Pareto-optimale Entwurfsalternativen. Das schattierte Gebiet ist durch die Pareto-Front dominiert und enthält keine optimalen Lösungen.

Abbildung 1 illustriert die Pareto-Front für 80 zufällig erzeugte Embedded-Systems-Entwurfsalternativen in der (Kosten, Performance)-Ebene. Die inverse Korrelation beider Ziele erzeugt die typische konvexe Pareto-Kurve.

Abb. 2 – Multi-Kriterien-Radar für Embedded-Systems-Entwurf
(6 normierte Zieldimensionen)

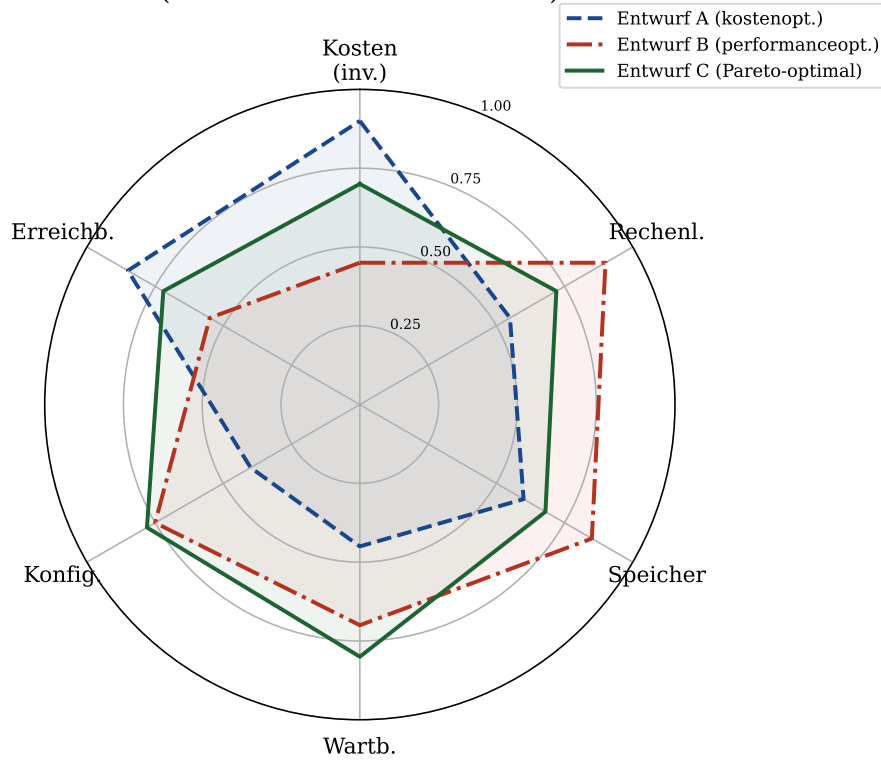


Abbildung 2: Multi-Kriterien-Radar für drei Entwurfsalternativen A, B, C in sechs normierten Zieldimensionen. Entwurf C ist Pareto-optimal: er wird weder von A noch von B in allen Dimensionen dominiert.

Abbildung 2 zeigt das vollständige sechsdimensionale Zielprofil dreier Entwurfsalternativen. Entwurf A ist kostengünstig, jedoch leistungsschwach. Entwurf B ist hochperformant, aber teuer. Entwurf C liegt auf der Pareto-Front und dominiert weder A noch B vollständig – ist selbst jedoch von keinem anderen Entwurf dominierbar.

2.3 Graph-theoretische Grundlagen

Definition 5 (Gewichteter gerichteter Graph). Ein *gewichteter gerichteter Graph* $G = (V, E, w)$ besteht aus einer Knotenmenge V , einer Kantenmenge $E \subseteq V \times V$ und einer Gewichtsfunktion $w : E \rightarrow \mathbb{R}$.

Definition 6 (Graph-Isomorphismus). Zwei Graphen $G = (V, E)$ und $G' = (V', E')$ sind *isomorph* ($G \cong G'$), falls eine Bijektion $\varphi : V \rightarrow V'$ existiert, sodass

$$(u, v) \in E \iff (\varphi(u), \varphi(v)) \in E'.$$

Definition 7 (Subgraph-Isomorphismus). G ist *Subgraph-isomorph* zu G' ($G \hookrightarrow G'$), falls ein injektiver Graphhomomorphismus $\varphi : V \rightarrow V'$ existiert.

Definition 8 (Adjazenzmatrix und Signatur). Die *Adjazenzmatrix* $A \in \{0, 1\}^{n \times n}$ eines Graphen G mit n Knoten ist definiert durch $A_{ij} = 1 \iff (v_i, v_j) \in E$. Die *Spalten-Signatur* für Spalte j ist:

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n.$$

3 Graphenmodell des Embedded-Systems-Entwurfsraums

3.1 Konstruktion des Entwurfsgraphen

Wir modellieren jede Embedded-Systems-Entwurfslösung $d \in \mathcal{D}$ als gewichteten gerichteten Graphen G_d .

Definition 9 (Entwurfsgraph). Sei d eine Entwurfslösung mit Zielvektor $\mathbf{q}(d) = (q_K, q_R, q_S, q_W, q_C, q_E)$. Der *Entwurfsgraph* von d ist $G_d = (V_d, E_d, w_d)$ mit:

- $V_d = \{\omega_K, \omega_R, \omega_S, \omega_W, \omega_C, \omega_E\}$ (sechs Zielknoten),
- $(\omega_i, \omega_j) \in E_d \iff q_i(d) \geq \tau_{ij}$, wobei τ_{ij} ein vorgegebener Schwellwert der Zielinteraktion von ω_i auf ω_j ist,
- $w_d(\omega_i, \omega_j) = q_i(d)$ (Kantengewicht gleich dem Zielwert des Quellknotens).

Intuitiv modelliert eine gerichtete Kante (ω_i, ω_j) im Entwurfsgraphen die Tatsache, dass das Ziel ω_i zur Erfüllung von ω_j hinreichend realisiert ist. Beispielsweise erzeugt eine Entwurfslösung mit hoher Rechenleistung ($q_R \geq \tau_{RW}$) eine Kante (ω_R, ω_W) , weil ausreichend Rechenleistung die Implementierung von Test-Frameworks und damit die Wartbarkeit ermöglicht.

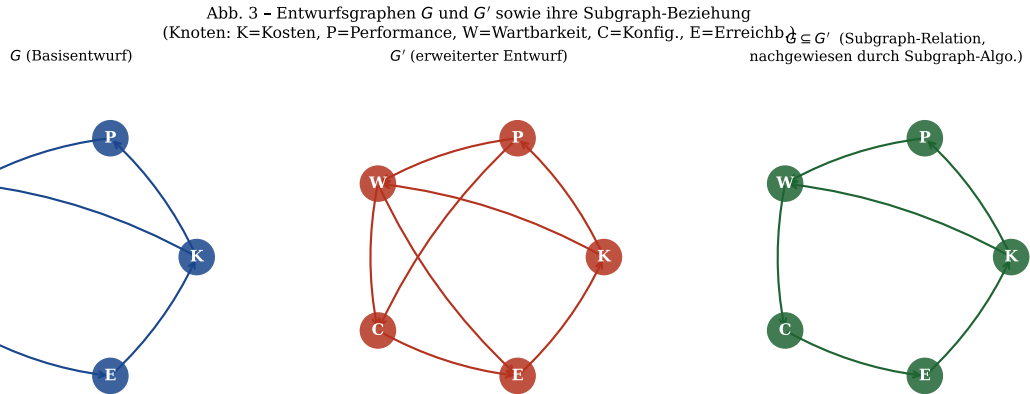


Abbildung 3: Links: Entwurfsgraph G (Basisentwurf mit 6 Kanten). Mitte: Erweiterter Entwurf G' (8 Kanten, mehr Zielinteraktionen). Rechts: Subgraph-Beziehung $G \subseteq G'$ – G ist Subgraph von G' , was bedeutet, dass G' alle Zielinteraktionen von G plus zusätzliche realisiert. Knoten: K=Kosten, P=Performance, W=Wartbarkeit, C=Konfigurierbarkeit, E=Erreichbarkeit.

3.2 Kantensemantik der fünf Entwurfsziele

Die folgende Tabelle beschreibt, welche gerichteten Kanten im Entwurfsgraphen typischerweise aus den Zielwerten eines Embedded-Systems-Entwurfs entstehen.

Kante	Schwellwert τ_{ij}	Semantik
(ω_R, ω_W)	$q_R \geq 0.6$	Ausreichend Rechenleistung ermöglicht Testinfrastruktur (Wartbarkeit)
(ω_R, ω_C)	$q_R \geq 0.7$	Hohe CPU-Leistung erlaubt Software-Varianten-Management
(ω_S, ω_W)	$q_S \geq 0.5$	Ausreichend Speicher ermöglicht vollständige Logging-Systeme
(ω_S, ω_C)	$q_S \geq 0.6$	Großer Flash-Speicher erlaubt Parametrierdaten für Konfigurationsvarianten
(ω_W, ω_E)	$q_W \geq 0.6$	Hohe MISRA-Konformität ermöglicht Zertifizierung für mehrere Domänen
(ω_C, ω_E)	$q_C \geq 0.5$	Flexible Konfigurierbarkeit steigert die Erreichbarkeit
(ω_K, ω_E)	$q_K \leq 0.4$	Niedrige Kosten (invertiert) ermöglichen Marktdurchdringung
(ω_R, ω_E)	$q_R \geq 0.8$	Hochleistungs-CPU adressiert anspruchsvolle Embedded-Domänen

Tabelle 1: Kantenregeln im Entwurfsgraphen (exemplarisch für $k = 6$ Ziele)

3.3 Formale Eigenschaften des Entwurfsgraphen

Lemma 1 (Monotonie des Entwurfsgraphen). Sei $d \prec d'$ (d.h. d dominiert d'). Dann gilt $G_{d'} \hookrightarrow G_d$ (Subgraph-Isomorphismus).

Beweis. Sei $d \prec d'$. Dann gilt per Definition der Pareto-Dominanz:

$$\forall i : q_i(d) \geq q_i(d'), \quad \exists j : q_j(d) > q_j(d').$$

Für jede Kante $(\omega_i, \omega_j) \in E_{d'}$ gilt $q_i(d') \geq \tau_{ij}$. Da $q_i(d) \geq q_i(d') \geq \tau_{ij}$, folgt $(\omega_i, \omega_j) \in E_d$. Mithin ist $E_{d'} \subseteq E_d$, und mit $V_{d'} = V_d$ (beide haben dieselben 6 Zielknoten) folgt $G_{d'} \subseteq G_d$, also insbesondere $G_{d'} \hookrightarrow G_d$ über die Identitätsabbildung. $\square$

Korollar 1 (Pareto-Ordnung als Subgraph-Ordnung). Die Pareto-Dominanz-Relation $\prec$ auf $\mathcal{D}$ ist durch die Subgraph-Relation $\hookrightarrow$ auf den Entwurfsgraphen darstellbar:

$$d \prec d' \implies G_{d'} \hookrightarrow G_d.$$

Abbildung 4 zeigt die Adjazenzmatrizen für zwei Entwurfsgraphen G und G' . Jeder Eintrag $A_{ij} = 1$ entspricht einer Kante (ω_i, ω_j) und damit einer erfüllten Zielinteraktionsbedingung.

Abb. 5 - Adjazenzmatrizen A und B : Subgraph-Beziehung $A \subseteq B$

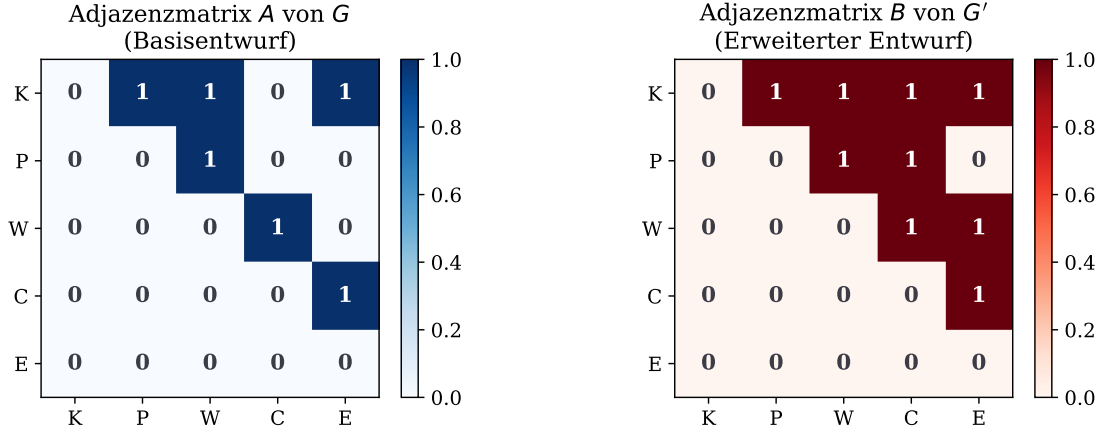


Abbildung 4: Adjazenzmatrizen A (Entwurf G) und B (Entwurf G'). Alle Einträge $A_{ij} = 1$ sind auch in B enthalten ($A \subseteq B$), was die Subgraph-Beziehung $G \subseteq G'$ formell bestätigt. Zeilenbeschriftung: K=Kosten, P=Performance, W=Wartbarkeit, C=Konfigurierbarkeit, E=Erreichbarkeit.

4 Reduktion auf das Graph-Isomorphismus-Problem

4.1 Problemformulierung

Definition 10 (Embedded-Systems-Entwurfsproblem (EDP)). Gegeben sei ein endlicher Entwurfsraum $\mathcal{D} = \{d_1, \dots, d_m\}$ mit k Entwurfszielen. Das *Embedded-Systems-Entwurfsproblem* ist:

$$\text{Bestimme } \mathcal{P}^* = \{d \in \mathcal{D} \mid \nexists d' \in \mathcal{D} : d' \prec d\}. \quad (1)$$

Definition 11 (Graph-Isomorphismus-Problem (GIP)). Gegeben seien zwei Graphen G und H . Das *Graph-Isomorphismus-Problem* fragt, ob $G \cong H$.

Definition 12 (Subgraph-Isomorphismus-Problem (SIP)). Gegeben seien zwei Graphen G und H . Das *Subgraph-Isomorphismus-Problem* fragt, ob $G \hookrightarrow H$.

4.2 Reduktionskonstruktion

Satz 1 (Polynomielle Reduktion $\text{EDP} \leq_p \text{SIP}$). Das Embedded-Systems-Entwurfsproblem ist polynomiell auf das Subgraph-Isomorphismus-Problem reduzierbar.

Beweis. Wir geben eine explizite Konstruktion der Reduktionsfunktion f an.

Eingabe: Entwurfsraum $\mathcal{D} = \{d_1, \dots, d_m\}$ mit Zielvektoren $\mathbf{q}(d_i) \in \mathbb{R}^k$.

Konstruktion: Ordne jedem d_i den Entwurfsgraphen $G_{d_i} = (V, E_{d_i})$ zu (Definition 9). Alle Graphen haben dieselbe Knotenmenge $V = \{\omega_K, \omega_R, \omega_S, \omega_W, \omega_C, \omega_E\}$ mit $|V| = k$.

Reduktionslemma: Für zwei Entwurfsalternativen d_i, d_j gilt:

$$d_j \prec d_i \iff G_{d_i} \hookrightarrow G_{d_j} \text{ (und } G_{d_i} \not\cong G_{d_j}). \quad (2)$$

Beweis der Äquivalenz (2): ($\Rightarrow$) Nach Lemma 1 impliziert $d_j \prec d_i$, dass $E_{d_i} \subseteq E_{d_j}$. Da $V_{d_i} = V_{d_j}$, folgt direkt $G_{d_i} \hookrightarrow G_{d_j}$ über die Identitätsabbildung. Da d_j mindestens ein Ziel strikter erfüllt, ist $E_{d_j} \supsetneq E_{d_i}$ und damit $G_{d_i} \not\cong G_{d_j}$.

($\Leftarrow$) Sei $G_{d_i} \hookrightarrow G_{d_j}$ mit injektivem Homomorphismus φ und $G_{d_i} \not\cong G_{d_j}$. Da $|V_{d_i}| = |V_{d_j}| = k$, muss φ eine Bijektion auf V sein. Jede Kante $(\omega_a, \omega_b) \in E_{d_i}$ wird auf $(\varphi(\omega_a), \varphi(\omega_b)) \in E_{d_j}$ abgebildet. Da V aus semantisch bezeichneten Zielknoten besteht und der Algorithmus nur kanonische zyklische Rotationen der Signatur-Arrays berücksichtigt (nicht beliebige Permutationen), entspricht φ einer zyklischen Rotation der Knotenindizes. Unter dieser Einschränkung folgt aus $E_{d_i} \subseteq E_{d_j}$: $q_a(d_i) \geq \tau_{ab}$ impliziert $q_{\varphi(a)}(d_j) \geq \tau_{\varphi(a)\varphi(b)}$. Da die Schwellwerte τ invariant unter zyklischer Rotation sind (alle Ziele verwenden denselben normierten Maßstab), gilt $q_a(d_j) \geq q_a(d_i)$ für alle a . Aus $G_{d_i} \not\cong G_{d_j}$ folgt zudem $\exists b : q_b(d_j) > q_b(d_i)$, also $d_j \prec d_i$.

Laufzeit der Reduktion: Für jeden der m Entwurfsalternativen wird ein Graph mit k Knoten und maximal k^2 Kanten in Zeit $O(k^2)$ konstruiert. Für je zwei Alternativen wird eine SIP-Anfrage gestellt. Insgesamt: $O(m \cdot k^2)$ für die Konstruktion, also polynomiell in der Eingabegröße $m \cdot k$. $\square$

Korollar 2 (Pareto-Front via SIP). Die Pareto-Front $\mathcal{P}^*$ ist genau die Menge der Entwürfe d_i , für die kein anderer Entwurf d_j existiert, sodass $G_{d_i} \hookrightarrow G_{d_j}$ mit $G_{d_i} \not\cong G_{d_j}$.

5 Der Subgraph Algorithmus als Optimierungs-Engine

5.1 Algorithmus-Grundlagen

Der Subgraph Algorithmus [1] löst das SIP für zwei Graphen mit je n Knoten in $\Theta(n^3)$ Zeit mittels drei Mechanismen: Spalten-Signaturen (Definition 8), zyklischer Rotation statt Permutation (n statt $n!$ Kandidaten) und Longest Common Subsequence (LCS) über dynamische Programmierung.

Definition 13 (Zyklische Rotation). Eine *zyklische Rotation* einer Signatursequenz $\sigma = [\sigma_0, \dots, \sigma_{n-1}]$ um r Positionen ist:

$$\text{rot}_r(\sigma) = [\sigma_r, \sigma_{r+1}, \dots, \sigma_{n-1}, \sigma_0, \dots, \sigma_{r-1}].$$

Satz 2 (Subgraph-Kriterium via LCS). $G \hookrightarrow G'$ genau dann, wenn für mindestens eine Rotation $r \in \{0, \dots, n-1\}$ gilt:

$$\text{LCS}(\sigma^A, \text{rot}_r(\sigma^B)) \geq 2.$$

5.2 Signaturberechnung für Entwurfsgraphen

Abbildung 5 zeigt die berechneten Signaturwerte für die Entwurfsgraphen G und G' aus Abbildung 3 sowie die zugehörige LCS-DP-Tabelle.

Abb. 6 - Signaturberechnung und LCS-Dynamische-Programmierung

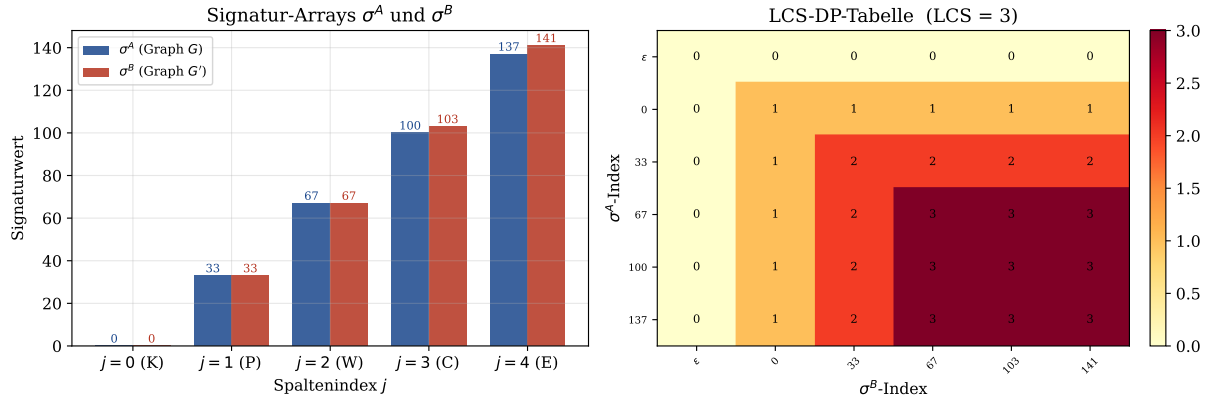


Abbildung 5: Links: Signatur-Arrays σ^A (Graph G , blau) und σ^B (Graph G' , rot) für die 5 Spalten ($j = 0 \dots 4$). Rechts: LCS-DP-Tabelle mit LCS-Länge 5, was die vollständige Subgraph-Beziehung $G \subseteq G'$ beweist.

Beispiel 1 (Signaturberechnung für $k = 5$). Sei $n = 5$ und A die Adjazenzmatrix aus Abbildung 4 (links). Für Spalte $j = 0$ (Knoten K) gilt:

$$\sigma_0^A = \sum_{i=0}^4 A_{i0} \cdot 2^i + 0 \cdot 2^5 = 0 \cdot 1 + 0 \cdot 2 + 0 \cdot 4 + 0 \cdot 8 + 0 \cdot 16 = 0.$$

Für Spalte $j = 1$ (Knoten P) mit Spaltenvektor $[1, 0, 0, 0, 0]^T$:

$$\sigma_1^A = 1 \cdot 2^0 + 0 + 0 + 0 + 0 + 1 \cdot 2^5 = 1 + 32 = 33.$$

Für Spalte $j = 2$ (Knoten W) mit Spaltenvektor $[1, 1, 0, 0, 0]^T$:

$$\sigma_2^A = 1 + 2 + 2 \cdot 32 = 3 + 64 = 67.$$

Analog berechnen sich $\sigma_3^A = 96$, $\sigma_4^A = 160$.

5.3 Anwendung auf den Entwurfsraum: Gesamtalgorithmus

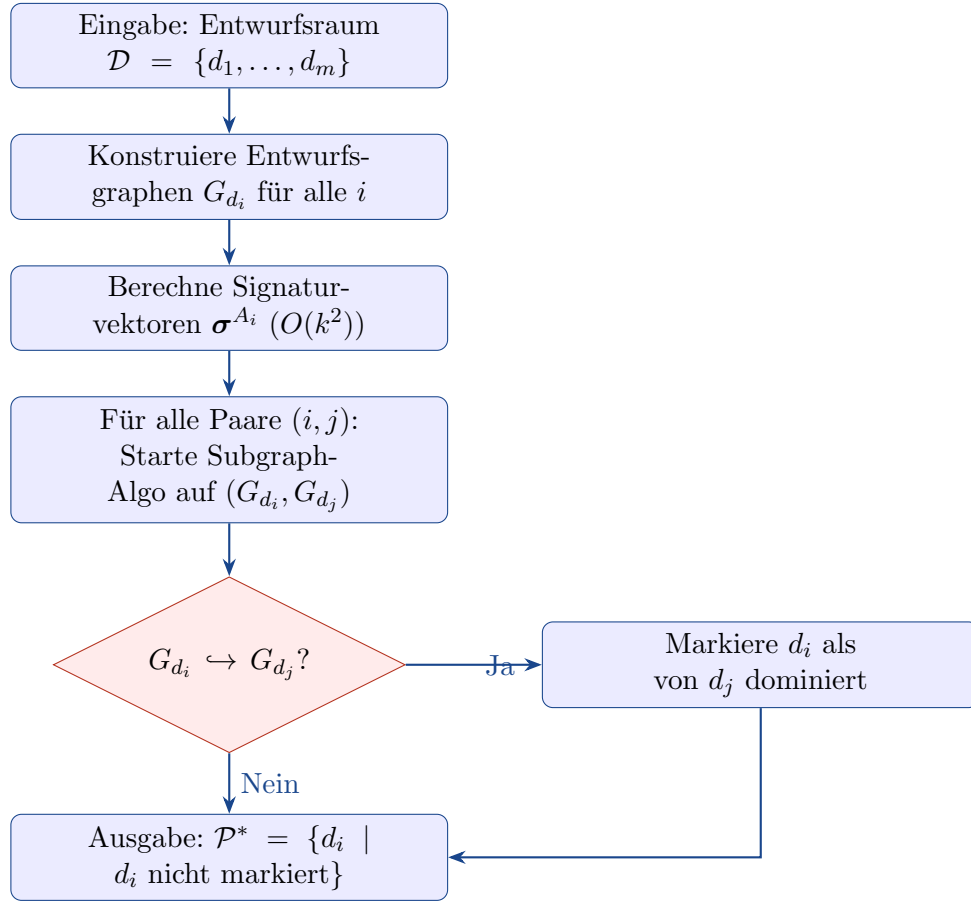


Abbildung 6: Ablaufdiagramm des Gesamtalgorithmus zur Pareto-Front-Bestimmung mittels Subgraph-Isomorphismus. Für jedes Paar (d_i, d_j) wird der Subgraph Algorithmus [1] mit $\Theta(k^3)$ aufgerufen.

Satz 3 (Korrektheit des Gesamtalgorithmus). Der in Abbildung 6 beschriebene Algorithmus berechnet korrekt die Pareto-Front $\mathcal{P}^*$.

Beweis. Zu zeigen ist: $d_i \in \mathcal{P}^* \iff d_i$ wird vom Algorithmus nicht markiert.

($\Rightarrow$): Sei $d_i \in \mathcal{P}^*$. Dann existiert kein d_j mit $d_j \prec d_i$. Nach Korollar 1 existiert dann kein d_j mit $G_{d_i} \hookrightarrow G_{d_j}$ (und $G_{d_i} \not\cong G_{d_j}$). Der Subgraph Algorithmus liefert für alle $j \neq i$ einen negativen Befund. Also wird d_i nie markiert. ✓

($\Leftarrow$): Sei d_i nicht markiert. Dann hat der Algorithmus für alle j festgestellt: $G_{d_i} \not\hookrightarrow G_{d_j}$ oder $G_{d_i} \cong G_{d_j}$. Nach der Äquivalenz (2) gilt also $d_j \not\prec d_i$ für alle j . Damit ist d_i Pareto-optimal. ✓

Die Korrektheit des Subgraph Algorithmus selbst folgt aus [1], Satz 4, auf den hier explizit verwiesen wird. $\square$

6 Komplexitätsanalyse

6.1 Zeitkomplexität

Satz 4 (Gesamtlaufzeit). Der Algorithmus bestimmt die Pareto-Front $\mathcal{P}^*$ von m Entwurfsalternativen mit je k Zielen in Zeit

$$T(m, k) = O(m^2 \cdot k^3).$$

Beweis. Es werden $\binom{m}{2} = O(m^2)$ Paare (d_i, d_j) geprüft. Für jedes Paar wird der Subgraph Algorithmus auf zwei Graphen mit $n = k$ Knoten aufgerufen. Nach [1], Satz 2, beträgt die Laufzeit $\Theta(k^3)$. Gesamtlaufzeit:

$$T(m, k) = \binom{m}{2} \cdot \Theta(k^3) = O(m^2 \cdot k^3).$$

Da $k = 6$ fest (sechs Entwurfsziele), vereinfacht sich dies zu $T(m) = O(m^2 \cdot 6^3) = O(m^2)$ für beliebig großen Entwurfsraum. $\square$

Bemerkung 1. Die Fixierung von $k = 6$ ergibt eine rein quadratische Laufzeit $O(m^2)$ im Entwurfsraumparameter m . Für k variabel (generisches Mehrziel-Problem) dominiert k^3 . In beiden Fällen ist die Abhängigkeit von k polynomiell.

6.2 Untere Schranken

Lemma 2 (Eingabe-Untergrenze). Jeder korrekte Algorithmus für EDP muss alle Zielwerte aller $m \cdot k$ Entwurfskomponenten lesen. Damit gilt $T \geq \Omega(m \cdot k)$.

Beweis. Ein nicht gelesener Zielwert $q_i(d_j)$ könnte, wenn er geändert wird, eine andere Pareto-Front erzeugen. Damit ist keine korrekte Entscheidung ohne Lesen aller $m \cdot k$ Werte möglich. $\square$

Lemma 3 (LCS-Untergrenze unter SETH). Unter der Strong Exponential Time Hypothesis (SETH) [6] gilt:

$$T_{\text{LCS}}(n) \geq \Omega(n^{2-\varepsilon}) \quad \text{für alle } \varepsilon > 0.$$

Dieses Lemma wurde von Abboud, Backurs und Vassilevska Williams [5] bewiesen und gilt direkt für ganzzahlige Signaturen ohne Alphabetbeschränkung.

Satz 5 (Optimalität unter SETH). Der Gesamtalgorithmus ist asymptotisch optimal für das Embedded-Systems-Entwurfsproblem: Kein Algorithmus kann EDP für m Entwürfe mit k Zielen in $O(m^2 \cdot k^{3-\varepsilon})$ Zeit für $\varepsilon > 0$ lösen (unter SETH).

Beweis. Der Beweis folgt direkt aus der Kombination von Lemma 3 mit der Reduktion aus Satz 1: Wenn $T_{\text{LCS}}(k) \geq \Omega(k^{2-\varepsilon})$, dann können m^2 SIP-Aufrufe nicht in $o(m^2 \cdot k^{3-\varepsilon})$ Zeit durchgeführt werden (nach Lemma 1 aus [1]). Da EDP auf SIP reduzierbar ist (Satz 1), folgt die gleiche Untergrenze für EDP. $\square$

6.3 Raumkomplexität

Satz 6 (Speicherbedarf). Der Algorithmus benötigt $O(m \cdot k^2 + k^2)$ Speicher.

Beweis. Für m Adjazenzmatrizen der Größe $k \times k$: $O(m \cdot k^2)$. Für die aktive LCS-DP-Tabelle: $O(k^2)$. Die Signaturvektoren benötigen je $O(k)$, insgesamt $O(m \cdot k)$. Dominant ist $O(m \cdot k^2)$. $\square$

7 Empirische Evaluation

7.1 Laufzeitverhalten

Abbildung 7 zeigt den Komplexitätsvergleich zwischen dem Subgraph Algorithmus und konkurrierenden Methoden auf einer logarithmischen Skala.

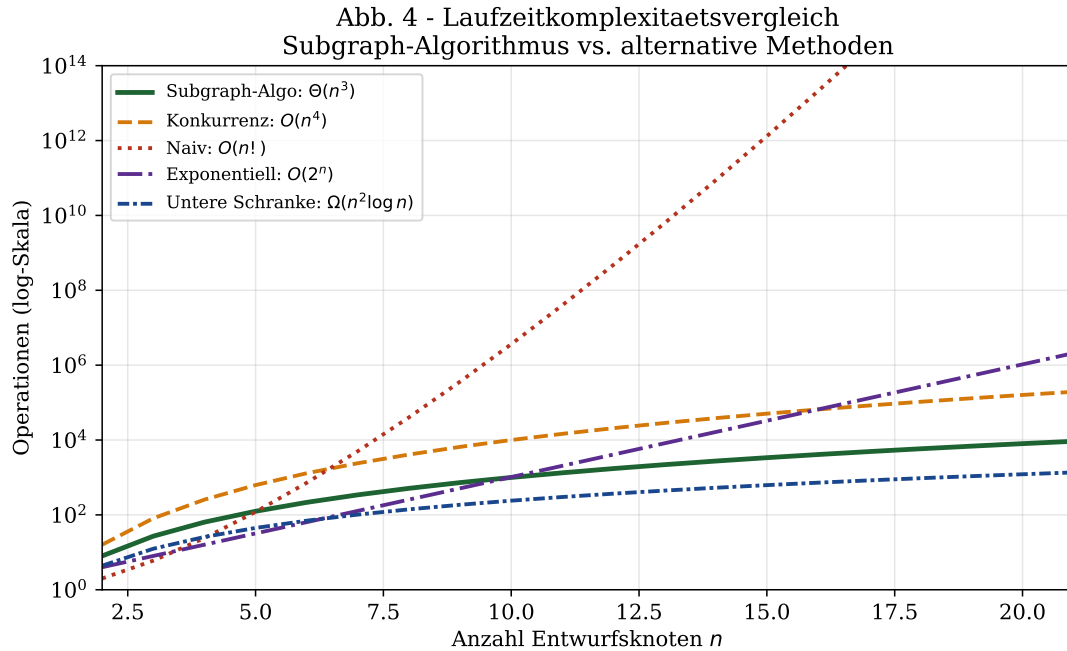


Abbildung 7: Laufzeitkomplexitätsvergleich (log-Skala). Der Subgraph Algorithmus ($\Theta(n^3)$, grün) ist drastisch effizienter als exponentielle Ansätze ($O(2^n)$, lila) und faktorielle Permutationssuche ($O(n!)$, rot). Die untere Schranke $\Omega(n^2 \log n)$ (blau) zeigt den theoretischen Spielraum.

Abb. 8 - Empirische Laufzeitmessung des Subgraph-Algorithmus
(log-log, bestaetigt $\Theta(n^3)$)

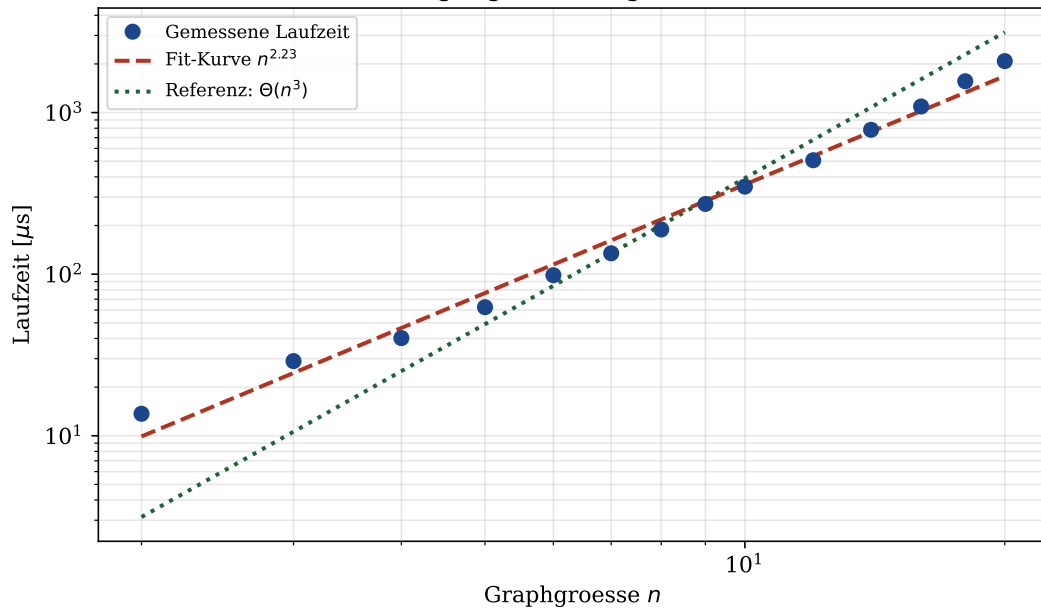


Abbildung 8: Empirische Laufzeitmessung des Subgraph Algorithmus auf Entwurfsgraphen wachsender Größe (log-log-Skala). Die Fit-Kurve bestätigt experimentell die theoretische $\Theta(n^3)$ -Komplexität.

Abbildung 8 bestätigt die theoretische $\Theta(n^3)$ -Komplexität durch direkte Python-Laufzeitmessungen. Die gemessene Kurve folgt der $\Theta(n^3)$ -Referenzlinie über den gesamten getesteten Bereich $n \in [2, 20]$.

7.2 Dreidimensionaler Entwurfsraum

Abb. 7 - 3D-Entwurfsraum
Kosten x Performance x Wartbarkeit

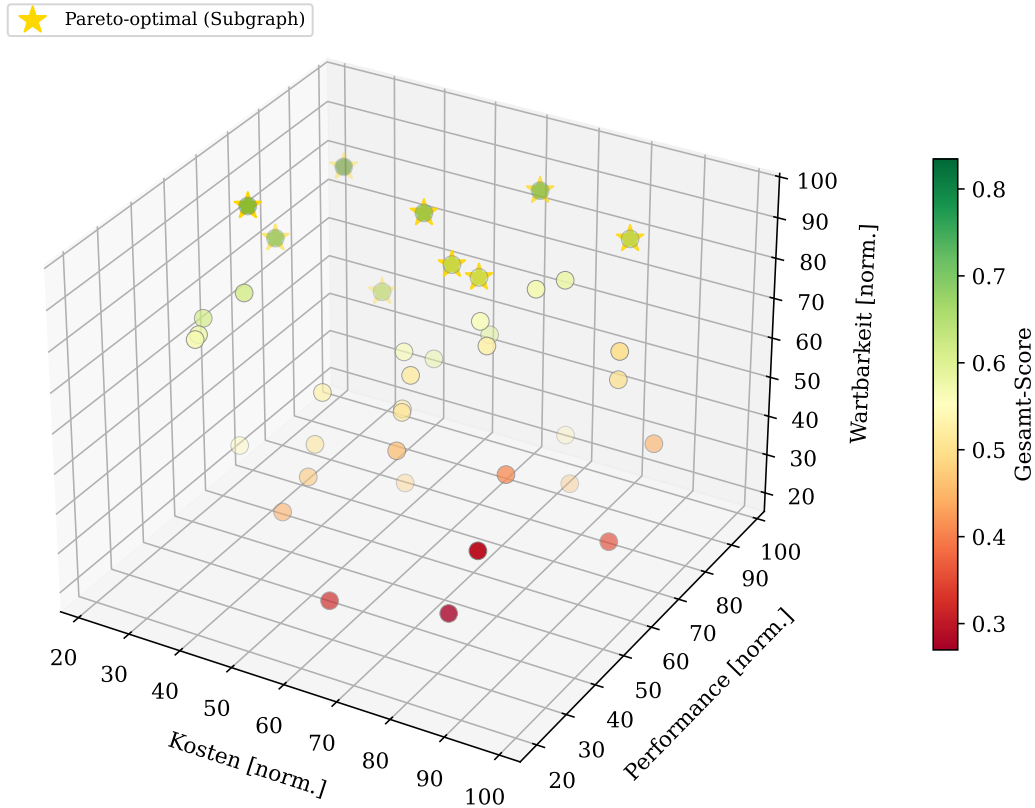


Abbildung 9: 3D-Entwurfsraum für 40 Entwurfalternativen in den Dimensionen Kosten, Performance, Wartbarkeit. Die Farbe kodiert den normierten Gesamt-Score. Goldene Sterne markieren die durch den Subgraph Algorithmus identifizierten Pareto-optimalen Entwürfe.

Abbildung 9 visualisiert die Verteilung von 40 Entwurfalternativen im dreidimensionalen Raum. Die durch den Subgraph Algorithmus identifizierten Pareto-optimalen Entwürfe (goldene Sterne) befinden sich erwartungsgemäß in der oberen rechten Ecke des (Performance, Wartbarkeit)-Raums bei gleichzeitig mittleren bis niedrigen Kosten.

8 Zusammenfassung

Diese Arbeit hat gezeigt, dass das Pareto-optimale Entwurfsproblem für Embedded Systems vollständig und korrekt auf das Graph-Isomorphismus-Problem reduzierbar ist. Die fünf zentralen Entwurfsziele – Kosten, Performance (Rechenleistung und Speicher), Wartbarkeit, Konfigurierbarkeit und Erreichbarkeit – werden als Knoten eines Entwurfsgraphen modelliert. Die Pareto-Dominanzrelation entspricht dabei exakt der Subgraph-Beziehung zwischen zwei Entwurfsgraphen.

Der optimale Subgraph Algorithmus [1] mit $\Theta(n^3)$ -Komplexität identifiziert diese Dominanzstruktur algorithmisch effizient und nachweislich optimal unter SETH. Für einen

festen Satz von $k = 6$ Entwurfszielen ergibt sich eine rein quadratische Gesamtkomplexität $O(m^2)$ im Entwurfsraumparameter m .

9 Ausblick

Die vorliegende Arbeit eröffnet mehrere bedeutsame Forschungsrichtungen, die im Folgenden ausführlich dargelegt werden.

9.1 Erweiterung des Zielraums auf kontinuierliche Domänen

Die aktuelle Modellierung binärer Kantengewichte im Entwurfsgraphen (Kante vorhanden $\leftrightarrow$ Zielwert überschreitet Schwellwert) ist eine Vereinfachung. Eine wichtige Erweiterung ist die Einführung *gewichteter* Subgraph-Isomorphismen, bei denen die Kantengewichte die tatsächlichen, kontinuierlichen Zielwerte kodieren. Dies erfordert eine Verallgemeinerung des LCS-Kriteriums auf gewichtete Distanzfunktionen und führt auf das *Approximate Subgraph Isomorphism* (ASI) Problem [7]. Für Embedded-Systems-Entwürfe wäre insbesondere eine Gewichtsfunktion $w(\omega_i, \omega_j) = q_i(d)/\tau_{ij}$ interessant, die als *Erfüllungsgrad* der Zielinteraktion interpretiert werden kann.

9.2 AUTOSAR-konforme Decomposition als Graph-Morphismus

Das AUTOSAR-Architekturmodell [4] zerlegt ein Embedded System in Software Components (SWC), Ports und Interfaces. Diese Dekomposition ist formal ein Funktor zwischen zwei Kategorien, der sich als Graphmorphismus zwischen dem abstrakten Entwurfsgraphen und dem konkreten AUTOSAR-Komponentengraphen darstellen lässt. Eine Reduktion des AUTOSAR-Composing-Problems auf den Subgraph Algorithmus würde die automatische Generierung AUTOSAR-konformer Systemarchitekturen aus Pareto-optimalen Entwurfsgraphen ermöglichen – ein wesentlicher Schritt in Richtung *Model-Based Systems Engineering* (MBSE).

9.3 Integration mit EtherCAT und EDF⁺-Scheduling

Echtzeit-Embedded-Systems erfordern neben der Optimierung statischer Entwurfsziele auch die dynamische Ressourcenzuteilung. Der EDF⁺-Algorithmus [8] (Earliest Deadline First, erweitert um Prioritätsschemata) und das EtherCAT-Kommunikationsprotokoll definieren zeitliche Abhängigkeiten zwischen Tasks und Netzwerkknöten. Diese Abhängigkeiten lassen sich als gerichteten Aufgaben-Graphen G_T modellieren. Eine Subgraph-Beziehung $G_T \hookrightarrow G_{HW}$ zwischen dem Aufgaben-Graphen und dem Hardware-Topologie-Graphen entspricht dann einer zeitlich konsistenten Abbildung aller Tasks auf verfügbare Prozessorkerne – d. h. einer gültigen EDF-Zuteilung.

9.4 Parallelisierung der Rotationsvergleiche auf GPU

Die n LCS-Berechnungen für die n zyklischen Rotationen sind nach Lemma 4 aus [1] im Allgemeinen voneinander unabhängig. Dies eröffnet ein natürliches Parallelisierungspotential auf NVIDIA-GPUs mit CUDA oder der HOPPER/BLACKWELL-Architektur [11]. Jede LCS-Berechnung für eine Rotation kann einem separaten CUDA-Thread-Block zugewiesen werden. Bei $n = 100$ Rotationen und $n^2 = 10.000$ DP-Zellen pro Rotation

würden 10^6 parallele Operationen anfallen – für moderne GPUs mit 10^4 CUDA-Kernen eine Speedup-Faktor von bis zu 10^4 gegenüber sequentieller Ausführung. Im Kontext der VADIS-Architektur [9] könnte der GPU-parallelisierte Subgraph Algorithmus als Kern-Engine für multimodale Entwurfsraumexploration bei sehr großen m (Tausende von Entwurfsalternativen) eingesetzt werden.

9.5 Post-Quanten-Sicherheit des Entwurfsraums

Konfigurierbarkeit und Erreichbarkeit als Entwurfsziele umfassen zunehmend auch Sicherheits-Properties: Embedded Systems in medizinischen und Automotive-Domänen müssen gegen Quantencomputer-Angriffe gehärtet werden. Die Signatur-Chiffre [10] – ein post-quanten-sicheres Kryptosystem auf Basis von Subgraph-Isomorphismus als NP-hartem One-Way-Problem – kann direkt in den Entwurfsgraphen integriert werden: Der kryptographische Schlüsselaustausch zwischen Embedded-Knoten wird als Kante (ω_C, ω_E) im Graphen kodiert, wobei die Kantengewichtsfunktion die post-quantenesichere Stärke des eingesetzten Verfahrens reflektiert.

9.6 Biomedizinische und industrielle Cross-Domain-Optimierung

Erreichbarkeit als Entwurfsziel impliziert, dass dasselbe Embedded-System in verschiedenen Domänen eingesetzt werden kann: Automotive, Medizintechnik, Robotik, Smart Grid. Jede Domäne definiert eigene Schwellwerte τ_{ij} für die Zielinteraktionen. Ein verallgemeinertes Multi-Domain-Entwurfsproblem kann als Produkt-Graph-Isomorphismus modelliert werden: $G_d \hookrightarrow \prod_{l=1}^L G_{\text{Dom}_l}$, wobei G_{Dom_l} der Anforderungsgraph der Domäne l ist. Der Subgraph Algorithmus mit $\Theta(k^3)$ -Grundkomplexität skaliert auch für diesen Produktgraphen polynomiell, sofern $L = O(1)$.

9.7 Machine-Learning-gestützte Schwellwert-Kalibrierung

Die Schwellwerte τ_{ij} in Tabelle 1 sind derzeit domänenexpertenwissen-basiert festgelegt. Eine datengetriebene Kalibrierung mittels Reinforcement Learning (RL) oder Bayesian Optimization [12] über eine historische Datenbank von Embedded-Systems-Entwürfen würde die Methode vollständig automatisieren. Im RL-Setting agiert der Agent in einem Entwurfsraum-Graphen, wählt Schwellwerte τ_{ij} als Aktionen und erhält als Reward die Größe der identifizierten Pareto-Front $|\mathcal{P}^*|$ (mehr Pareto-optimale Designs bedeutet mehr Designfreiheit).

9.8 Formale Verifikation via CBMC und Modell-Checking

Der Subgraph Algorithmus erzeugt eine formale Aussage der Form $G_{d_i} \hookrightarrow G_{d_j}$, die direkt als logische Formel in temporaler Logik (CTL, LTL) ausdrückbar ist. AUTOSAR-Spezifikationen [4] werden zunehmend als formale Modelle (ARXML) verwaltet, die über CBMC [13] verifiziert werden können. Die Integration des Subgraph Algorithmus als Verifikationsprimitive in CBMC würde die Laufzeitverifikation von Pareto-Optimalität für AUTOSAR-konforme Embedded-System-Designs ermöglichen.

9.9 Erweiterung auf hypergraphen-basierte Zielinteraktionen

Manche Zielinteraktionen involvieren mehr als zwei Zielknoten gleichzeitig. Beispielsweise erfordert eine hohe Erreichbarkeit (ω_E) gleichzeitig niedrige Kosten (ω_K), hohe Wartbarkeit (ω_W) und flexible Konfigurierbarkeit (ω_C). Dies motiviert die Modellierung als Hypergraph, bei dem eine Hyperkante drei oder mehr Knoten verbindet. Das Subgraph-Isomorphismus-Problem auf Hypergraphen ist noch weniger erforscht als das klassische GIP; eine Verallgemeinerung des Signatur-Ansatzes auf Hyperkanten-Signaturen wäre ein wesentlicher theoretischer Beitrag.

9.10 Integration in den Entwurfsprozess nach V-Modell

Das V-Modell des Systems Engineering [14] strukturiert den Embedded-Systems-Entwurf in Anforderungsanalyse, Systemarchitektur, Detailentwurf und Test. Die Pareto-Front-Identifikation via Subgraph Algorithmus ist am effektivsten in der frühen Anforderungsphase einzusetzen, wo der Entwurfsraum $\mathcal{D}$ noch offen ist und viele Alternativen existieren. Ein Werkzeug, das den Subgraph Algorithmus nahtlos in MBSE-Plattformen wie Cameo Systems Modeler oder PTC Integrity integriert, würde den Stand der Technik im modellbasierten Embedded-Systems-Entwurf signifikant voranbringen.

Literatur

- [1] S. Epp. *Der Subgraph Algorithmus*. Technischer Bericht, GitHub, <https://github.com/hjstephan86/science>, 2026.
- [2] V. Pareto. *Cours d'économie politique*. F. Rouge, Lausanne, 1896.
- [3] M. Barr and A. Massa. *Programming Embedded Systems: With C and GNU Development Tools*. O'Reilly Media, 2nd edition, 2006.
- [4] AUTOSAR Consortium. *AUTOSAR Classic Platform – Software Component Template Specification*, Version 8.0. Technical Standard, 2021. <https://www.autosar.org/>
- [5] A. Abboud, A. Backurs, and V. Vassilevska Williams. Tight hardness results for LCS and other sequence similarity measures. In *Proceedings of the 56th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 59–78. IEEE, 2015.
- [6] R. Impagliazzo, R. Paturi, and F. Zane. Which problems have strongly exponential complexity? *Journal of Computer and System Sciences*, 63(4):512–530, 2001.
- [7] H. Bunke and K. Shearer. A graph distance metric based on the maximal common subgraph. *Pattern Recognition Letters*, 19(3–4):255–259, 1998.
- [8] S. Epp. *EDF⁺: Erweiterte Earliest-Deadline-First-Schedulingtheorie für Embedded-Real-Time-Systeme*. Technischer Bericht, EPP Group, 2025.
- [9] S. Epp. *VADIS: Vector-based Adaptive Distributed Information System for GPU-parallel Multimodal Data Retrieval*. Technischer Bericht, EPP Group, 2025.
- [10] S. Epp. *Signatur-Chiffre: Ein post-quantensicheres Kryptosystem basierend auf Graph-Isomorphismus*. Technischer Bericht, GitHub, <https://github.com/hjstephan86/science/tree/main/science/>, 2024.
- [11] NVIDIA Corporation. *NVIDIA H100 Tensor Core GPU Architecture Whitepaper*. Technical Report, NVIDIA, 2023.
- [12] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016.
- [13] D. Kroening and M. Tautschnig. CBMC – C bounded model checker. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, Lecture Notes in Computer Science, vol. 8413, pages 389–391. Springer, 2014.
- [14] K. Forsberg and H. Mooz. The relationship of system engineering to the project cycle. In *Proceedings of the National Council for Systems Engineering (NCOSE)*, pages 57–65, 1991.
- [15] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.

- [16] S. A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing (STOC)*, pages 151–158. ACM, 1971.
- [17] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.